

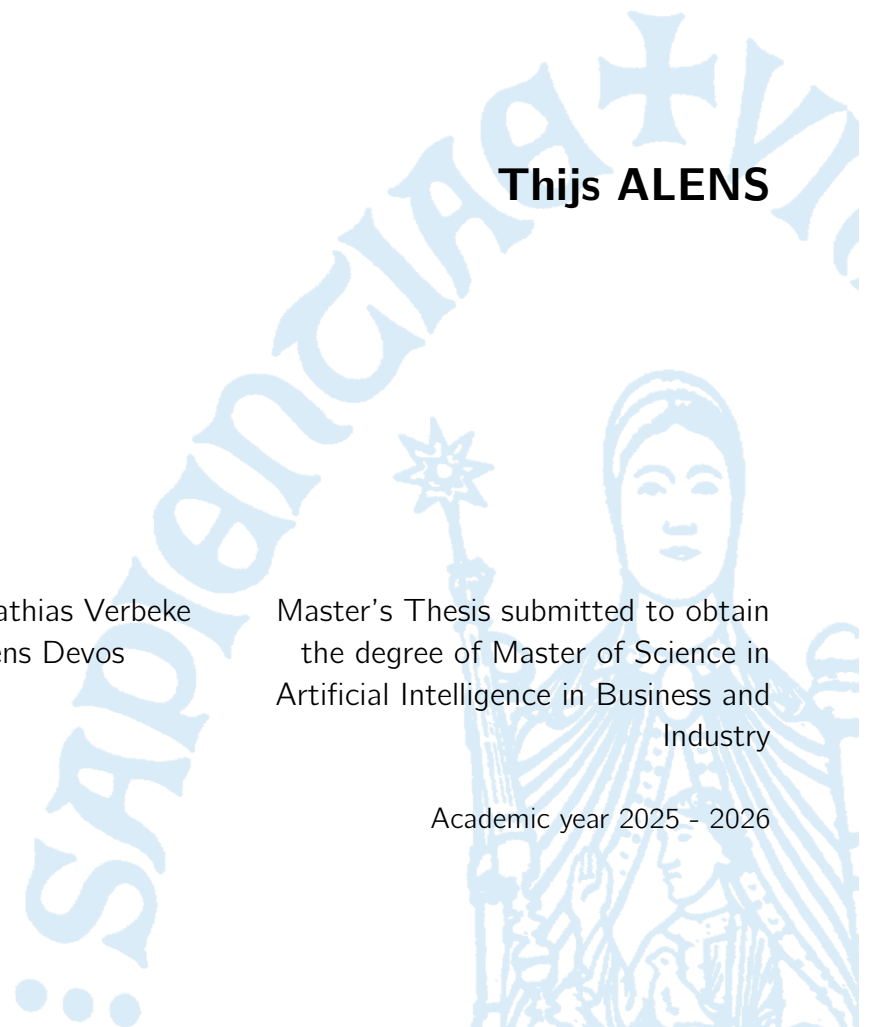
Modeling object localization uncertainty for robot grasping

Thijs ALENS

Supervisor(s): Prof Dr Mathias Verbeke
Dr Ir Laurens Devos

Master's Thesis submitted to obtain
the degree of Master of Science in
Artificial Intelligence in Business and
Industry

Academic year 2025 - 2026



©Copyright KU Leuven

This master's thesis is an examination document that has not been corrected for any errors.

Without written permission of the supervisor(s) and the author(s) it is forbidden to reproduce or adapt in any form or by any means any part of this publication. Requests for obtaining the right to reproduce or utilise parts of this publication should be addressed to KU Leuven - Bruges, Spoorwegstraat 12, B-8200 Bruges, +32 50 66 48 00 or via e-mail fet.bruges@kuleuven.be.

A written permission of the supervisor(s) is also required to use the methods, products, schematics and programs described in this work for industrial or commercial use, for referring to this work in publications, and for submitting this publication in scientific contests.

Preface

Het voorwoord vul je persoonlijk in met een appreciatie of dankbetuiging aan de mensen die je hebben bijgestaan tijdens het verwezenlijken van je masterproef en je hebben gesteund tijdens je studie.

In the preface, you write a personal account with an acknowledgement or expression of gratitude to the people who assisted you in completing your master's thesis and supported you during your studies.



*Copyright Information:
student paper as part of an academic education
and examination.
No correction was made to the paper
after examination.*

Samenvatting

De (korte) samenvatting, toegankelijk voor een breed publiek, wordt in het Nederlands geschreven en bevat **maximum 3500 tekens**. Deze samenvatting moet ook verplicht opgeladen worden in KU Locket.

Summary

This (short) summary, accessible to a wide audience, is written in English and contains a **maximum of 3500 characters**. This summary must also be uploaded to KU Locket.

Keywords: Enter (more or less) five keywords.

Contents

1	Introduction	2
1.1	Positioning	2
1.2	Problem statement	2
1.3	Objectives	3
2	Literature study	5
2.1	Uncertainty in AI	5
2.1.1	Aleatoric uncertainty	5
2.1.2	Epistemic uncertainty	6
2.1.3	Sources of uncertainty	7
2.2	Quantifying uncertainty	8
2.2.1	Aleatoric uncertainty	8
2.2.2	Epistemic uncertainty	8
2.3	Uncertainty-aware AI models	9
2.3.1	Bayesian Neural Networks	10
2.3.2	Deep Ensemble Methods	11
2.3.3	Monte Carlo dropout	13
3	Dataset	14
3.1	Setup	14
3.1.1	Realsense camera	14
3.1.2	Table and robot arm	14
3.2	Labeling	15
3.2.1	Pixel coordinates	16
3.2.2	Real-world coordinates	17
3.2.3	Robot's position	18
3.3	Capturing and using data	19

3.3.1	Collecting data	19
3.3.2	PyTorch Dataset	19
3.3.3	Data split	20
4	Models for uncertainty aware object detection	21
4.1	The structure of an uncertainty aware model	21
4.2	Training and evaluating uncertainty aware models	22
4.3	Data scaling	23
4.4	Deep ensembles	24
4.4.1	Simple convolutional neural network (CNN)	24
4.4.2	Pretrained ResNet[15]	25
4.5	Monte Carlo (MC) dropout	26
5	Experiments	28
5.1	Evaluation metrics	28
5.1.1	Loss Function	28
5.1.2	Uncertainty intervals	29
5.1.3	Classifying predictions based on error and uncertainty	30
5.1.4	Error-uncertainty graph	31
5.2	Experiment 1: Evaluating on a test set	32
5.3	Experiment 2: Evaluating on out-of-distribution data	33
5.3.1	Multiple objects in the scene	33
5.3.2	Difficult lighting	33
5.3.3	Human intervention	33
6	Discussion	34
6.1		34
7	Conclusion	35
7.1	Future work	35
8	GenAI code of conduct for students	38

List of Figures

2.1	The blue line represents a model which is not well generalized, and thus captures the noise in the data. The red line represents a model which is well generalized, and thus captures the general distribution of the data.	9
3.1	The setup used.	15
3.2	Annotating the pixel coordinates of the objects.	16
3.3	The chessboard pattern used for calibration.	17
3.4	An example of a depth image with its corresponding RGB image captured by the Realsense camera.	20
4.1	A residual block, the main building block of ResNet.	25
5.1	An example of an error-uncertainty graph.	31
5.2	Error-uncertainty graphs for the Deep Ensemble models on the test set. . . .	32

List of Tables

4.1	Comparison of different architectures for the simple CNN model. “Lr” stands for learning rate, “Br” stands for bagging ratio, “m” stands for the number of models in the ensemble, and “Loss” stands for the ensemble validation loss.	25
4.2	Comparison of different architectures for the ResNet model. “Lr” stands for learning rate, “Br” stands for bagging ratio, “m” stands for the number of models in the ensemble, “PW” stands for pretrained weights used, and “Loss” stands for the ensemble validation loss.	26
4.3	Comparison of different architectures for the MC dropout model. “DR” stands for dropout rate at each layer, “Lr” stands for learning rate, “Br” stands for bagging ratio, “F” stands for the number of forward passes, and “Loss” stands for the model’s validation loss.	27
5.1	Comparison of the different models on the test set.	32

Acronyms

AI artificial intelligence. 2, 3, 5, 9, 18

CNN convolutional neural network. viii, 5, 6, 9, 24, 26

CV computer vision. 2

DNN deep neural network. 5, 6, 11, 12, 24

FN false negative. 30

FP false positive. 30

GAN generative adversarial network. 6, 9

MC Monte Carlo. viii, x, 9, 13, 26, 27

MLP multilayer perceptron. 6, 9

NLL negative log-likelihood. 23

NN neural network. 10, 12, 13, 22

PoV point of view. 15

TN true negative. 30

TP true positive. 30

Chapter 1

Introduction

1.1 Positioning

We live in a world where automation is being adopted at a rapidly increasing pace. Luggage is sorted and autonomously transported across airports, autonomous vacuum cleaners are used daily, medicines are produced automatically with extreme cleanliness, etc. This trend also applies to robotics, with robots becoming increasingly prevalent in factories and warehouses to improve efficiency, quality, and safety.

To operate properly, these robots require sophisticated systems, which are increasingly based on artificial intelligence (AI). More specifically, computer vision (CV) is a subdomain of AI that enables computers to interpret and make decisions based on visual data from cameras. This technology can be used to detect and localize objects, enabling the robot to process them. Additionally, more general AI can be used to plan the robot's actions and control its actuators to perform desired tasks. Furthermore, systems can be trained to monitor processes, detect defects in finished products, or predict maintenance needs. The use of AI thus not only brings greater efficiency and quality gains but also enables robots to execute much more complex tasks.

These real-world scenarios require AI systems to be highly robust and reliable. However, AI systems often struggle when faced with new or unknown situations, leading to unreliable predictions. Therefore, systems should be able to express uncertainty about their predictions. This capability enables safety mechanisms to prevent the robot from taking actions based on uncertain predictions. Uncertainty-aware AI models achieve this by outputting both a prediction and a corresponding uncertainty score.

1.2 Problem statement

While an AI system offers many advantages in a factory setting, it also introduces new challenges that must be addressed. One of the issues with these systems is that they will

always provide an answer, even when they are unsure of it. Consequently, if the system is uncertain about its output, it will not inform the user and will continue operating normally. This can sometimes be beneficial, as the system might still make the correct decision, thereby increasing efficiency compared to a human worker who would need to pause and assess the new situation. However, it is more likely to be detrimental, as incorrect decisions can lead to product defects, damage to the robot or its environment, or harm to nearby humans.

A system might be unsure about its output if it needs to infer information from data that was not (properly) represented during training. Most training data is gathered in controlled environments where all conditions are optimal. However, the system's performance can significantly worsen in real-world scenarios that differ from the training environment. These differences can include varying lighting conditions due to shadows or reflections, camera lens distortion from dust, or sudden changes in illumination. Other unexpected events, such as a human operator placing the wrong object or the sudden movement of an already placed object, can also disrupt the AI system.

1.3 Objectives

This thesis aims to address these issues by developing and evaluating uncertainty-aware regression models which infer the center (x and y coordinates) of an object. This so a robot arm can grasp them, if the uncertainty is low, otherwise avoid taking actions to keep the environment safe. Our focus is on making these models work in challenging real-world conditions.

The primary research question is: How can we develop uncertainty-aware object localization models for a robot arm, that can effectively identify and communicate uncertainty in their predictions, particularly in challenging real-world conditions?

This overarching question is further explored through the following sub-questions:

- What uncertainty aware (object localization) models already exist, and how do they perform in real-world conditions?
- How can we effectively quantify and communicate uncertainty in object localization predictions to ensure safe robot operation?

To address these questions, the following objectives are set:

- Conduct a comprehensive literature review on uncertainty and existing uncertainty-aware models with an emphasis on object localization.
- Create a dataset where the center of an object (x and y coordinate) is annotated. This in both normal and challenging real-world conditions.
- Develop, implement and evaluate uncertainty-aware models to predict the center of an object along with an uncertainty score.

- Design and implement safety mechanisms that utilize the uncertainty information to prevent the robot from taking actions based on uncertain predictions.

Chapter 2

Literature study

In this chapter, we discuss existing research relevant to uncertainty in AI models and how it maps to real-world scenarios. First, we elaborate on what uncertainty is and the different ways it can sneak into AI models. Next, we discuss how this uncertainty can be quantified. Finally, we explore existing methods to make AI models uncertainty-aware and discuss how these concepts apply to a regression problem.

2.1 Uncertainty in AI

Many complex systems need to be modeled and understood. These systems can be modeled using AI models, which offer a way to grasp their complexity and operate them as efficiently and safely as possible. However, like humans, these models are not perfect and can make mistakes. When using standard, non-uncertainty-aware AI models, these mistakes are not communicated to the user. For example, when a CNN is used to detect an object on a conveyor belt, it outputs the predicted class without indicating its certainty about the prediction. If the object is only partially visible, the lighting is poor, or the object closely resembles another, the model might be highly uncertain but will still output a class as if it were confident. All possible uncertainties regarding deep neural networks (DNNs) can be categorized into two groups: aleatoric uncertainty and epistemic uncertainty [1] [2].

2.1.1 Aleatoric uncertainty

Aleatoric uncertainty is a type of uncertainty that arises from noise inherent in the data. Its nature is irreducible, meaning this uncertainty will remain present regardless of how much data is gathered.

Measurement errors When gathering data, mistakes can be made during the measuring process. These often stem from limitations of the measurement devices or imprecise data

labeling. It is important to note that, if kept subtle, these mistakes can actually serve as a way to regularize a network.

As an example, we consider a regression problem where a model predicts the x and y coordinates for the center of an object. The camera used for data capture might have a low resolution, and thus a lack of pixels which could cause an object to appear differently. Furthermore, it could lead to inaccurate annotations; the center might be labeled as (x_1, y_1) when it is actually $(x_1 + \delta, y_1 + \epsilon)$, with δ and ϵ representing small measurement errors.

2.1.2 Epistemic uncertainty

Epistemic uncertainty originates from noise within the model or its training process. This can stem from various causes, such as the model's architecture, the training procedure, or a lack of knowledge due to insufficient or irrelevant training data. In theory, its nature is reducible, meaning that gathering better data or using a better suited model architecture can decrease this uncertainty. However, doing so in practice carries a significant computational cost, making it not always feasible to eliminate this uncertainty.

Variability in real world situations The real world changes constantly, creating diverse settings where objects may behave or look differently. Ideally, all potential situations should be sufficiently covered by the training set. If not, the network trained on this data cannot guarantee accurate outputs for new real-world situations. A distribution shift occurs when these real-world scenarios differ from the training data. These shifts are highly challenging for a DNN to process, causing its performance to fluctuate significantly.

Continuing with the regression problem of predicting an object's center. When a DNN is trained on a dataset featuring optimal lighting and no occlusions, no other objects covering the view, etc., it will perform well in that specific setting. However, during actual inference, conditions might change unexpectedly, such as altered lighting, the appearance of new objects, etc. Because the dataset lacks examples of these new conditions, the trained DNN fails to adapt, leading to a drop in performance.

Architecture specification errors Network structures differ significantly, with each architecture having specific advantages and disadvantages regarding performance and prediction uncertainty.

Suppose a factory requires the previously discussed regression model to meet their needs. Numerous model architectures are available, such as a multilayer perceptron (MLP), a CNN, a U-net [3], a generative adversarial network (GAN), or a YOLO network [4], etc. Each model has its own configuration of layers, connections, and activation functions. Consequently, all these networks will have their own capabilities and thus uncertainty for a given task.

Training procedure errors During model training, there are many decisions that need to be made. These include selecting the learning rate, the number of epochs, the data batching strategy, the optimizer, the weight initialization, etc. Each combination of these hyperparameters results in a different model with its own specific errors and uncertainty.

For instance, the dataset for object center regression can be split into different batch sizes, ordered in various ways, etc. The chosen architecture can be trained using these diverse data handling methods, using different parameters to optimize it. This again leads to varying models that perform better in certain areas but poorly in others.

Errors by unknown data Once fully trained, the model may encounter entirely unprecedented situations during inference. Although unexpected within the original problem scope, the model must process these situations, resulting in high uncertainty.

In the factory example, the model is trained for a specific environment and objective: detecting object centers in an industrial setting. If an unexpected object, such as a bird, enters the camera's viewpoint, the model will be highly uncertain because it has never seen one before. This occurs not because the environment was insufficiently taught to the model, but due to the completely unexpected nature of the bird being in the factory.

2.1.3 Sources of uncertainty

In our specific problem of object localization for robotic arm grasping, both aleatoric and epistemic uncertainties are present. We would like to tackle both!

First is the aleatoric uncertainty, inherent in the data, primarily caused by measurement errors. While we will attempt to minimize these errors, they will inevitably remain present in the dataset. However, as mentioned in Section 2.1.1, these errors are not the primary concern; if kept subtle, they can act as a form of model regularization.

One of the sources of aleatoric uncertainty are the errors introduced during the data collection and annotation process. More concretely, when indicating the center of an object in the image, the annotator will not be able to perfectly identify the center, leading to small errors in the labels. Furthermore, the centers of the objects are most conveniently labeled in pixel-space, but the model should predict the centers in real-world-space, which requires a transformation that can introduce some errors as well. How to deal with the errors associated with the labeling process will be discussed further in Section 3.2.

Secondly, there is the epistemic uncertainty, which needs to be taken care of. This uncertainty is inherent to the model and its training process, and thus needs to be tackled by the architecture and behavior of the model itself. How this is precisely done will be discussed in Section 2.3.

2.2 Quantifying uncertainty

Models who can handle and quantify the uncertainties as described in 1.1 are very sought-after in safety-critical applications like robot arm grasping. For our problem, it is very important to know what factors can cause the model to be uncertain about its predictions and how to quantify this. When using a model in practice, we are interested in the uncertainty on the prediction of the model, also called predictive uncertainty. This is depended on two previously mentioned things: the uncertainty of the data (data uncertainty), which falls under the aleatoric uncertainty, and the uncertainty of the model (model uncertainty), which falls under the epistemic uncertainty [1].

2.2.1 Aleatoric uncertainty

When we want to quantify the aleatoric uncertainty, we need to capture how mistakes can be made during the capturing and processing of the data. The conventional way of doing this is by using a well generalized model, which is trained on a large dataset. This makes the model learn the general distribution of the data. If the model is not well generalized, it learns the noisy data without real knowledge about the more general distribution of the data. Figure 2.1 shows an example of a model which is not well generalized, and thus captures the noise in the data. How we now quantify the aleatoric uncertainty is by looking at the loss of the model on the validation data. If the model is well generalized, the loss on the validation data will be low, and thus the aleatoric uncertainty will be low as well. The exact opposite is true for a model which is not well generalized, as it will have a high loss on the validation data, and thus a high aleatoric uncertainty. However, these losses are very dependent on the loss function used during training, which is not ideal if we want to compare different models with different loss functions.

A model can also be designed to directly predict the aleatoric uncertainty, as a second output, in addition to the main prediction. This requires a modification of the loss function to account for the uncertainty output. This is more complex to implement, but allows the model to directly learn the aleatoric uncertainty from the data. It is thus dependent on the data rather than the loss function, which is more ideal. How this is done in practice will be discussed in Section 4.2.

2.2.2 Epistemic uncertainty

Quantifying epistemic uncertainty is a bit more complex to tackle, as it requires multiple models, hyperparameters, different usage of data etc. to be tested. There are different methods to account for this uncertainty, especially in theory. First of all, we can use a Bayesian approach to model the uncertainty of the model parameters (see 2.3.1). This is a very principled way to model the uncertainty, as it needs an infinite amount of models to capture the uncertainty of the model parameters, which is not possible in practice. However,

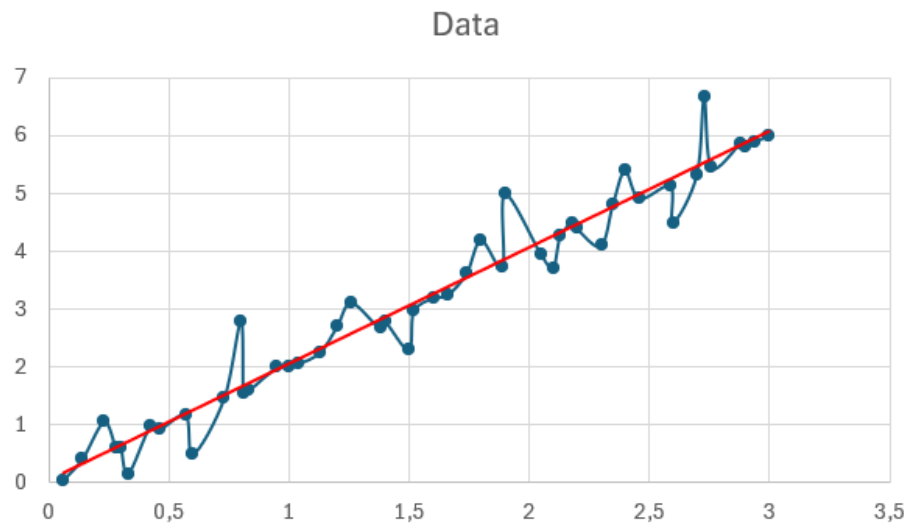


Figure 2.1: The blue line represents a model which is not well generalized, and thus captures the noise in the data. The red line represents a model which is well generalized, and thus captures the general distribution of the data.

its idea perfectly captures the way epistemic uncertainty establishes itself. Another way to model the uncertainty of the model parameters is by using an ensemble of models (see 2.3.2). This is a more practical way to model the uncertainty, as it is an approximation of the Bayesian approach, using a finite number of models to capture the uncertainty of the model parameters. A third way to model the uncertainty of the model parameters is by using dropout during inference (see 2.3.3). MC dropout, like the ensemble method, is more practical than the Bayesian approach. It keeps dropout turned on during inference, which allows it to capture the uncertainty of the model parameters by generating different outputs for the same input, which can then be combined to get a final output and uncertainty score.

2.3 Uncertainty-aware AI models

“Conventional” AI models (MLPs, CNNs, GANs, YOLO’s [4], etc.) are not designed to be uncertainty-aware, meaning they do not have a way to quantify the uncertainty of their predictions. This makes that new methods had to be developed to make AI models uncertainty-aware. The ones we will be discussing, all have their foundation in the bayesian way of thinking, which is a very principled, mathematically-sound way to model uncertainty.

2.3.1 Bayesian Neural Networks

A neural network (NN) is a function which maps an input $x \in X$ to an output $y \in Y$ using a model f_θ , as shown in Equation (2.1).

$$y = f_\theta(x) \quad (2.1)$$

In our case, the input vector x is an image of an object, and the output vector y is the predicted coordinates of the center of the object. The parameters θ of the model are learned during the training phase, where the model is trained on a dataset consisting of input-output pairs.

Aleatoric uncertainty [2] For regression problems, the most conventional way to predict targets is to let the model directly predict the targets using Equation (2.1) directly. Its parameters θ are learned by minimizing a loss function (like the L^2 or L^1 loss) on the training data. However, this does not allow the model to capture the aleatoric uncertainty of the data, as it only predicts a single value for each target, without any indication of how certain it is about this prediction. Instead of predicting a single value for each target, the model can be designed to predict a distribution over the targets as showed by Equation (2.2a). By assuming a Gaussian distribution for the targets, the model can be designed to predict the mean μ and variance σ^2 of the distribution, as shown in Equation (2.2b).

$$p(y|x, \theta) = \mathcal{N}(y; \mu_\theta(x), \sigma_\theta^2(x)), \quad (2.2a)$$

$$f_\theta(x) = [\mu_\theta(x), \log \sigma_\theta^2(x)]^T \quad (2.2b)$$

The variance σ^2 captures the aleatoric uncertainty of the data, while the mean μ captures the prediction of the model. The model can be trained by minimizing the negative log-likelihood of the data under the predicted distribution, as shown in Equation (2.3).

$$\min(-\sum_i \log(p(y_i|x_i, \theta))) \quad (2.3)$$

Epistemic uncertainty [2][5] To capture the uncertainty in the model parameters θ , we need the Bayesian approach, otherwise referred to as Bayesian inference. In the Bayesian approach, we place a prior distribution over the model parameters, which captures our beliefs about the parameters before seeing any data. Using the data, we can then compute the posterior distribution over the model parameters θ (given in Equation (2.4)), which captures our updated beliefs about the parameters after seeing the data.

$$p(\theta|X, Y) = \frac{p(Y|X, \theta)p(\theta)}{p(Y|X)} \quad (2.4)$$

$p(\theta)$ is the prior distribution over the model parameters, which can be chosen based on our beliefs about the parameters. When there are no prior beliefs of the model's parameters, its most commonly chosen to use a Gaussian distribution. However, its choice can be a way to regularize the model [6]:

- L^2 regularization is equivalent to using a Gaussian prior.
- L^1 regularization is equivalent to using a Laplace prior.

$p(Y|X, \theta)$ is the likelihood of the data given the model parameters, which captures how well the model fits the data. This is calculated by passing the whole dataset through the model with a softmax output layer and calculating the probability of the data under the predicted distribution.

$p(Y|X)$ is the evidence, which is a normalizing constant that ensures that the posterior distribution is a valid probability distribution. It is calculated by integrating the likelihood over all possible values of the model parameters, as shown in Equation (2.5).

$$p(Y|X) = \int p(Y|X, \theta)p(\theta)d\theta \quad (2.5)$$

However, computing the evidence is not feasible in practice, as it requires integrating over all possible values of the model parameters, of which a DNN has many. As by consequence, the posterior distribution is also not feasible to compute, as it requires computing the evidence. This is why we need to use approximations to compute the posterior distribution.

2.3.2 Deep Ensemble Methods

A deep ensemble method [7] rests on the idea that a combination of (simpler) models can be trained and perform better than a single (complex) model. Not only that, but it also outputs a great uncertainty score about its predictions. A typical ensemble method uses multiple decision trees as their models, a deep ensemble method uses, as the name suggests, uses DNNs as its models. Creating a deep ensemble method contains 3 main steps:

- A proper scoring rule, which is used as a way to train the models in the ensemble.
- Adversarial training, which is used to make the models in the ensemble more robust.
- The actual ensembling, which combines the outputs of the different models to create a final output and uncertainty score.

The scoring rule A scoring rule is a way to measure the quality of a model's probabilistic predictions. It should encourage the model to generate outputs which are accurate and are capable of telling when it is uncertain about its output. To make this a bit more concrete, let's take our use case of predicting the center of an object:

- A great scoring rule should punish the model hard if it predicts coordinates with a low uncertainty, when in fact the ground truth states the center is somewhere else completely.
- The model should be encouraged to predict coordinates with a high uncertainty if these coordinates are far from the ground truth.
- If the model predicts coordinates which are close to the ground truth with a high uncertainty, it should be punished.
- When the model predicts coordinates which are close to the ground truth with a low uncertainty, it should be rewarded.

For regression problems, we can adapt the negative log-likelihood loss function to be a proper scoring rule for our two-part output (see Section 4.2).

Adversarial training Adversarial training is a technique which is used to make NNs more robust against small changes which can drastically change the output of the model. An adversarial example is a modified version of an input image that is designed to fool the model into making a mistake. Take our case as an example, when coordinates of an object are predicted with a high probability, it could be that with a very small change to the image, the network all of a sudden thinks it is somewhere completely different with high confidence. This is something we want to avoid at all cost, since this big swing eliminates any notion of uncertainty the model could have had.

Adversarial training tries to fix these big swings by generating these adversarial examples during training and adding them to the training set. Further following the example, an adversarial example is generated from a correctly predicted input image and added to the training set with the same label as the original image. By doing this, the model learns to be more robust against these small changes, and the big swings in output are reduced.

Ensembling The final step in creating a deep ensemble method is the actual ensembling. The paper [7] suggests to create an ensemble of 5 DNNs to predict an output with an uncertainty score. Each model in the ensemble should be trained independently, however this does not mean they should be different model architectures. In fact, when randomly initializing the NN parameters, along with shuffling the training data, the networks were different enough to obtain good results. This makes that no methods like bagging or boosting are needed to create diversity in the ensemble. Which means all the training data can be used for the training of the models in the ensemble, which is beneficial for the DNNs which need more data to train properly.

To get the final output and uncertainty score, the outputs of the different models are combined. Each model in the ensemble outputs a probability distribution over the different classes. These probability distributions are then averaged to get the final output distribution. The label with the highest probability in this final distribution is taken as the final output

with the uncertainty belonging to it. Lets continue the example of classifying images of disks and cubes to make this more concrete. Assuming we have an ensemble of 3 models, and they output the following probability distributions for a given input image:

- Model 1: [0.7 (disk), 0.3 (cube)]
- Model 2: [0.4 (disk), 0.6 (cube)]
- Model 3: [0.9 (disk), 0.1 (cube)]

Averaging these distributions results in the final output distribution: [0.67 (disk), 0.33 (cube)]. The final output is thus 'disk' with an uncertainty score of 0.67.

2.3.3 Monte Carlo dropout

Dropout is a regularization technique commonly used in the training of NNs to prevent overfitting. The idea is to randomly drop some neurons in the network during the training phase. This forces the model to use all the available neurons and not rely too much on a small subset of them. When the model is fully trained and ready for inference, dropout is usually turned off, using all the neurons to make predictions.

MC dropout [8] expands on this idea by keeping dropout turned on during inference. Every time a set of neurons is dropped, the network changes a bit, which can result in different outputs for the same input. Each alteration of the network can be seen as a different model in an ensemble. By letting the model make multiple forward passes with dropout turned on, we can gather a set of different outputs for the same input with their own predicted probability distributions. These outputs can then be combined in a similar way as the deep ensemble method in Section 2.3.2 to get a final output and uncertainty score.

Reusing the example of classifying images of disks and cubes, assuming we make 3 forward passes with MC dropout, we could get the following outputs:

- Forward pass 1: [0.8 (disk), 0.2 (cube)]
- Forward pass 2: [0.5 (disk), 0.5 (cube)]
- Forward pass 3: [0.6 (disk), 0.4 (cube)]

Averaging these distributions results in the final output distribution: [0.63 (disk), 0.37 (cube)]. The final output is thus 'disk' with an uncertainty score of 0.63.

Chapter 3

Dataset

In this chapter, we will discuss the dataset created for training and evaluating the uncertainty-aware object localization models. We will outline the requirements for the dataset based on the problem statement and objectives defined in Chapter 1. Furthermore, we will describe the data collection process, including the setup and annotation used.

3.1 Setup

The setup we used is shown in Figure 3.1, where we have a realsense camera mounted on a pole in front of a table with a robot arm on it. The view of the camera is a bit wider and longer than the table, but only the part of the table within the black border lines is usable by the robot.

3.1.1 Realsense camera

Realsense camera's are a type of camera which capture both RGB and depth information, the one we used specifically is the "Realsense Depth Camera D415". The realsense camera's have good support in the form of RealSense SDK [9]. It is a library which supports different languages like Python, C#, ROS, etc. and makes it easy to use the Realsense camera's. We will use its Python binding, which is called "pyrealsense2". It allows us to easily setup and handle both the RGB and depth camera streams independently from each other. It also contains information about the intrinsic parameters of the camera, these embody the focal length and the principal point of the camera. Its importance will become clear in Section 3.2.2.

3.1.2 Table and robot arm

Table The table is used as support for the the camera, robot arm and objects. As Figure 3.1 shows, the table is equipped with a grid of black lines in the shape of a rectangle and a black

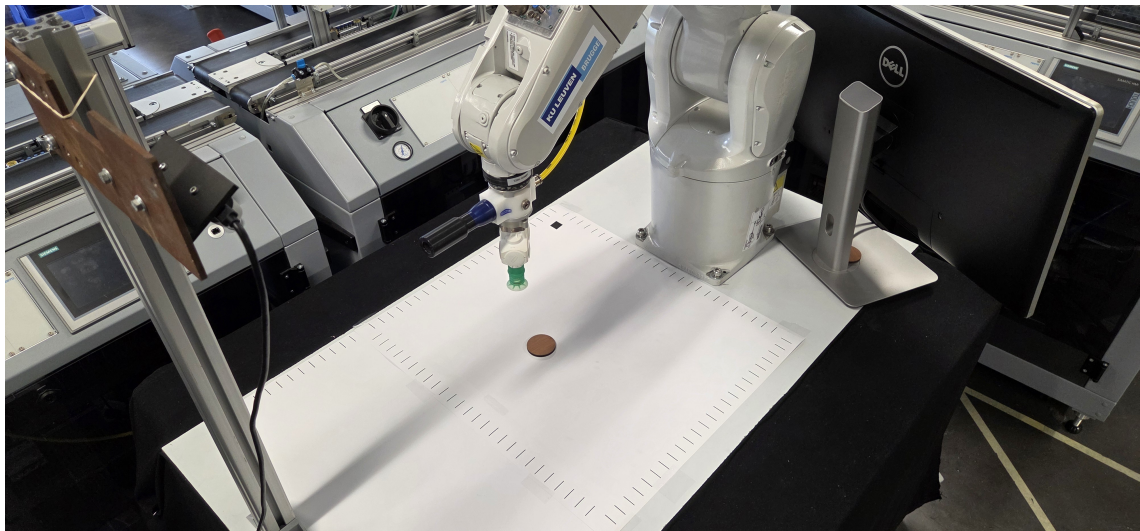


Figure 3.1: The setup used.

square. Each line is 2cm apart from each other and is used as a reference so we can capture the real-world coordinates of the objects. The black square is used as the origin of this maden coordinate system, more specifically the bottom right corner of the square is the origin. The x-axis goes horizontally (relative to the camera's point of view (PoV)) to the right, the y-axis goes vertically downwards. There is no z-axis defined as all the objects are roughly the same height; consequently, the robot arm only needs to know the x and y coordinates of the objects to be able to grasp them. More information about the specific use of the grid will be given in Section 3.2.2.

Robot arm The robot arm used in our setup is the "ABB IRB 1200" [10]. It is equipped with a vacuum gripper, which is used to grasp the objects on the table. The robot arm is controlled by a computer, which sends commands to the robot arm to move to specific coordinates and to open and close the gripper. Previous work with this specific robot arm [11] will be used to control it.

3.2 Labeling

We needed to create a dataset for regression of the position of the objects on the table. There where three approaches we could take to do this:

- We could use the pixel coordinates.
- We could use the real-world coordinates.
- We could use the robot's position.

There are some advantages and problems with all three approaches.

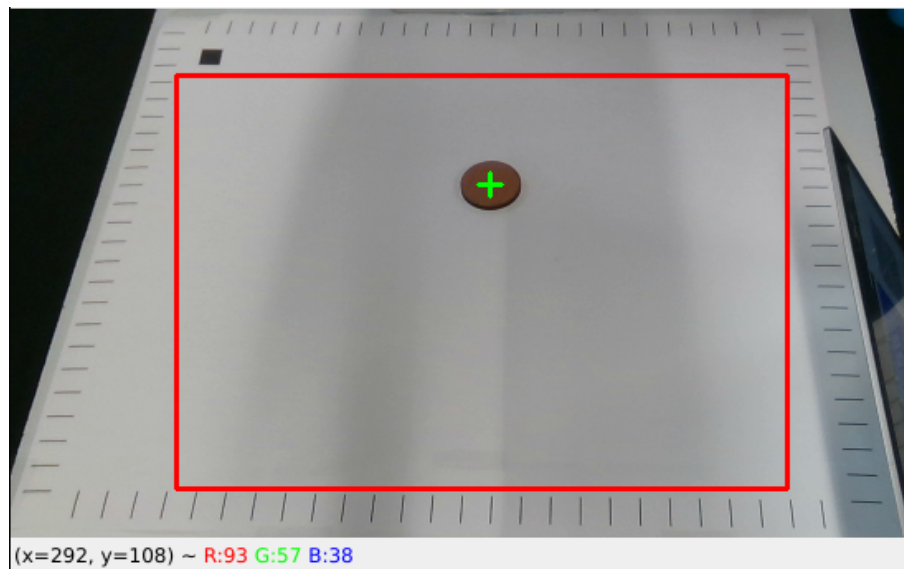


Figure 3.2: Annotating the pixel coordinates of the objects.

3.2.1 Pixel coordinates

Pixel coordinates are coordinates in the image plane of the camera, they are defined by the number of pixels in the x and y direction. A computer sees an image as a 2D array of pixels per colour channel, so for RGB images, there are 3 2D arrays in total, one for each colour channel. Conventionally, the origin of the pixel coordinates is the first value in the array ($img_r[0][0]$), the x-axis goes horizontally to the right and the y-axis goes vertically downwards. The libraries most commonly used for computer vision, like OpenCV [12] and PyTorch [13], use pixel coordinates for their operations on images, which makes it very convenient to use and incooperate. It makes it easy to annotate the dataset, as we can create a simple script to click on the objects in the images and save the pixel coordinates of the clicks as annotations, as shown in Figure 3.2. Even more, using some of the tools provided by the OpenCV [12] library, we can even automate this clicking process. Its resulting annotation should be more accurate and consistent than a user clicking on the center of the object, purely visually. However, it is not 100% consistent, as some objects were not detected and thus still required some manual clicking.

This way of annotating in pixel space, requires some post-processing to achieve what we want, which is the real-world coordinates of the object so the robot arm can use them to grasp the objects. This transformation requires information about the characteristics of the camera, called the intrinsic parameters, as well as the position and orientation of the camera relative to the table, which are called the extrinsic parameters. The intrinsic parameters of the camera we are using, are provided by the Realsense SDK [9] and should be very accurate, as they are measured and provided by the manufacturer. The extrinsic parameters need to be calibrated for each specific setup, they consist of a rotation and translation from the camera's coordinate system (pixel coordinates) to the table's coordinate system. OpenCV [12] provides



Figure 3.3: The chessboard pattern used for calibration.

a convenient way to do this calibration, this being the “findChessboardCorners” function. It detects the corners of a chessboard pattern, as shown in Figure 3.3, these corners can then be used to calculate the rotation and translation vector using the “solvePnP” function, which is also provided by OpenCV [12]. Once the parameters are obtained, we can use them to transform the pixel coordinates of the objects to real-world coordinates. However, this process is not perfect and can introduce some errors, which can be a source of uncertainty in the dataset, which is something we want to avoid as much as possible.

3.2.2 Real-world coordinates

We presume real-world coordinates as the coordinates measured in the physical world, in our case they are relative to an origin defined on the table. Since this way of doing the coordinates uses real measurements, it is more intuitive and directly usable for the robot arm. However, it comes with an important caveat, as it is more difficult to annotate data efficiently. We need to define an origin for the coordinate system, as well as the x and y axis, and then we need to measure the coordinates of the objects relative to this origin. This

measuring can be done with a ruler, but this will be too time-consuming and cause too much human error, which can introduce uncertainty in the dataset which we want to minimize. A better option is to use a predefined grid on the table, which we could detect and use to measure the coordinates of the objects. This is better, but we can not draw a grid on the table, as we would like the AI to do inference without any outside assistance (like a grid). We thus need a way to create a “virtual” grid which can easily be removed from the images, but can still be used to measure the coordinates of the objects. This is where the black lines around the table come in.

They could be detected by the camera [14] with the help of the OpenCV [12] library and used to create this virtual grid by connecting opposite lines. However, this did not work well, as not all the lines were detected. This is most probably due to the lighting not being consistent enough for the filters used to detect the lines to work properly. An other problem is the resolution of the camera being too low to detect the individual lines, this was most prominent in the corners of the table where the lines are closest to each other. We ended up using a more manual approach, where we manually annotated the pixel coordinates of the center of the lines as well as the origin square. This introduces some human error, but still ensured we got a consistent and accurate enough grid to measure the real-world coordinates of the objects.

All this setting up of the grid, is done virtually before any labeling is done and saved as a reference for the labeling process. When using the system to either label the dataset or to do inference, the image is cropped so the black lines are not visible anymore. Now we have a way to measure the real-world coordinates of the objects without introducing too much human error, while still being able to remove the grid from the images so the AI can do inference without any outside assistance.

3.2.3 Robot’s position

Ultimately, the robot arm needs to move itself to the correct position to be able to grasp the objects. This means that we could also use the robot’s position as labels for the dataset. However, this, just as the real-world coordinates, is not that easy to annotate, as we need to move the robot arm to the correct position and then read out its coordinates. There is no easy trick (like using OpenCV [12] for real-world coordinates) to do this fast, accurately but most of all consistent. We thus would need to do manually move the robot to the correct location for each data point.

This takes a lot of time and introduces some human error and is not that consistent, so we opted against using the robot’s position as labels for the dataset.

```
{
  "pixel": {
    "x": 221,
    "y": 62
  },
  "world": {
    "x": 288.09246369430474,
    "y": 134.27617670846834
  }
}
```

Listing 1: Example of the JSON file structure.

3.3 Capturing and using data

3.3.1 Collecting data

To actually capture the data, we created a script which implements both the pixel-coordinates and real-world coordinates approach, this so we can choose which one to ultimately use for our models. It is an interactive script, which shows the RGB and depth images from the camera in real-time and allows us to capture the data by pressing a button. When the button is pressed, the script tries to automatically detect the center of the object. The user can then either accept the detected coordinates or manually click on the center of the object to correct the coordinates. After pinpointing the object's center, the script crops the image so the calibration lines are not visible anymore and saves the RGB and depth images. Following this, it calculates the real-world coordinates of the object and saves them along the pixel-coordinates in a JSON file, which is structured as shown in Listing 1.

Each trio of RGB image, depth image and JSON file is considered one data point in the dataset, all with a unique identifier.

3.3.2 PyTorch Dataset

Since we are using the PyTorch [13] framework for the training and evaluation of our models, we needed a way to easily load and preprocess the data in a way that is compatible with it. PyTorch [13] provides a "Dataset" class, which is an abstract class that we can inherit from and implement our own dataset.

We kept two options open when creating this class, as we wanted to keep as much options open as possible to differentiate between the different approaches.

- Have a 3D tensor as input, which is the RGB image, or have a 4D tensor as input, which is the RGB image concatenated with the depth image.

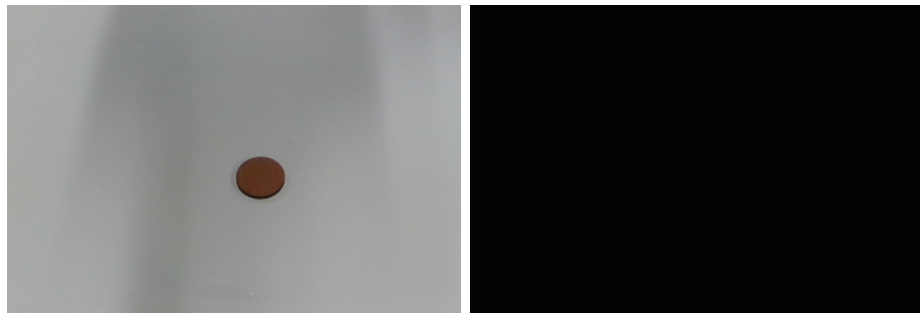


Figure 3.4: An example of a depth image with its corresponding RGB image captured by the Realsense camera.

- Use the pixel coordinates as labels, or use the real-world coordinates as labels.

When looking at the depth images (of which an example is shown in Figure 3.4) captured by the Realsense camera, we noticed that they did not contain that much information about the object, as the objects are pretty small in height.

3.3.3 Data split

Now that we have the data required, we need to split it into a training and validation set. These need to stay independent from each other, as we want to evaluate the model's performance on data it has never seen before. We also wanted to keep a separate test set, which is used to compare the different models on data similar to the training data, but which is still independent from the training and validation sets. A common split is 80% for training, 10% for validation and 10% for testing, so this is what we ended up using as well.

Chapter 4

Models for uncertainty aware object detection

In this chapter, we will discuss the different uncertainty-aware models discussed in Chapter 2 and compare their ability to capture and communicate uncertainty. We will first discuss the way an uncertainty aware model is trained and evaluated, after which we will discuss the different models in more detail ending with a comparison of the different models for our use case.

4.1 The structure of an uncertainty aware model

An uncertainty aware model differs from a normal model, in the sense that it needs to output its uncertainty alongside the prediction.

The model's outputs a number of values, depending on what the application requires: If we want our model to output a single value, for example predicting the time it takes for a robot to move to a certain point, in a conventional model, it would output a value in seconds, for example 5.2 seconds. However, in an uncertainty aware model, it would output two values, for example 5.2 seconds and 0.8 seconds. The first value is the prediction, and the second value is the uncertainty of that prediction. In our case, we want to predict the position of an object in 2D space, so we want to output the x and y coordinates of the object, along with the uncertainty of those predictions. This means that our model will output four values, for example 57.2, 33.4, 0.8, 0.6. The first two values are the predicted x and y coordinates in mm, and the last two values are the uncertainty of those predictions also in mm.

The way of computing the uncertainty of a prediction depends on the kind of uncertainty we want to cover.

For aleatoric uncertainty, the uncertainty linked to the data, we can use a single model with two outputs. The model will learn the distribution of training data and, when trained properly, will be able to tell when something is off. In practice, we train a model with two

outputs (one extra for every predicted value) the conventional way with a loss-function and back-propagation. How this is done in detail, will be discussed in Section 4.2.

Epistemic uncertainty, the uncertainty associated with the model parameters, is a bit more complex. It requires multiple models, according to bayes rule (Section 2.3.1) even all possible models, to capture the uncertainty. The idea of communicating the uncertainty in practice, is very related to aleatoric uncertainty, as it also outputs two values (one extra per predicted value), one prediction and an uncertainty value. However, the uncertainty value is no longer a value that the individual models need to learn, rather its the “agreement or disagreement” between the models. This “agreement or disagreement” can be interpreted in different ways depending on the application of the model. For regression tasks, the most obvious choice is the variance of the predictions, but it can also be an interval in which the predictions fall, or even a distribution where the user can ask for a specific confidence level. For classification tasks, the most obvious choice is the entropy of the predictions, but it can also be the variance of the predicted probabilities. Since we are using a regression model, we will focus on the different options for this type of model.

The nice thing about the way uncertainty aware models are structured, is the fact that this is exactly the same for both aleatoric and epistemic uncertainty. A single model, which can only capture aleatoric uncertainty, will output a value for the prediction along with a value for the uncertainty. Multiple models combined to form a model which can capture epistemic uncertainty, will output a value for the prediction along with a value for the uncertainty. The difference lies in the way the uncertainty is interpreted, for a single model, the uncertainty is a value that the model needs to learn, for multiple models, the uncertainty is a value that is derived from the agreement or disagreement between the models.

4.2 Training and evaluating uncertainty aware models

Training an uncertainty aware model differs quite a bit from training a conventional NN. This because of the nature of the models, wanting to communicate uncertainty along with the prediction. We thus need a way to train the model to include uncertainty in its predictions, and we also need a way to evaluate the model on its ability to capture and communicate uncertainty. There are a few “new” ways of thinking about when a model is doing what it is supposed to do, and when it is not:

- If the model is sure about its prediction, and the prediction is correct, great! The model is doing its job.
- If the model is sure about its prediction, but the prediction is wrong, then the model is not doing its job as it is unable to communicate its faults.
- If the model is unsure about its prediction, and the prediction is wrong, then the model is doing its job as it is able to communicate its uncertainty about its prediction.

- If the model is unsure about its prediction, but the prediction is correct, then the model is not doing its job as it is unable to communicate its confidence in its prediction.

We need a way so these cases can be captured in a loss function, so that the model can learn to do what it is supposed to do. Fortunately, there already exist such a loss function, Gaussian negative log-likelihood (NLL) loss [7] (Equation (4.1)).

$$\mathcal{L} = -\log(p_\theta(y_n|\mathbf{x}_n)) = \frac{\log(\sigma_\theta^2(\mathbf{x}))}{2} + \frac{(y - \mu_\theta(\mathbf{x}))^2}{2\sigma_\theta^2(\mathbf{x})} + \text{constant} \quad (4.1)$$

Where $\mu_\theta(\mathbf{x})$ is the predicted value, $\sigma_\theta^2(\mathbf{x})$ is the predicted uncertainty, and y is the true value. In our case, y_n is a 2D vector containing the x and y coordinates of the object, $\mu_\theta(\mathbf{x}_n)$ and $\sigma_\theta^2(\mathbf{x}_n)$ are also 2D vectors containing the predicted x and y coordinates and the predicted uncertainty of the object, respectively. These vectors do not change anything fundamental to the loss function. This loss function captures the cases we discussed above, as it penalizes the model for being sure about a wrong prediction, and it also penalizes the model for being unsure about a correct prediction.

4.3 Data scaling

During the training of the models below, we came across an issue related to the data. More specifically, the mismatching of the data's order of magnitude and the model's weights' order of magnitude. The data we gathered is in mm, so the values of the x and y coordinates are fairly big hovering between 10 and 500. The weights of the model, on the other hand, are initialized with small values, and during the training, they do not change much in terms of order of magnitude. This means that the model needs to somehow find a way to scale to match the order of magnitude of the outputs, which is very challenging.

To solve this issue, we started by scaling the outputs of the model by a 10^{-3} factor, this so the model's outputs are somewhat in the same order of magnitude as the model's weights. This helped a lot, but is not the most elegant solution, as this factor is arbitrarily chosen and is not based on any theoretical foundation.

A more elegant way of calculating this factor, is to use the data itself to calculate it. We can calculate the mean and standard deviation of the x and y coordinates in the training data, and use these to normalize the data. The scaling factor can be calculated as Equation (4.2), which centers the data so it follows a standard normal distribution with zero mean and unit variance. Again, note the vectors in the equation, as we have two values for the x and y coordinates.

$$\mathbf{z} = \frac{\mathbf{y} - \boldsymbol{\mu}}{\boldsymbol{\sigma}} \quad (4.2)$$

This way, the model's outputs are in the same order of magnitude as the model's weights, and the model can learn to predict values that are in the same order of magnitude as the

data. After the forward pass of the model, we can then unscale the outputs to get the actual predictions in mm.

4.4 Deep ensembles

A deep ensemble, as explained in Section 2.3.2, is an ensemble of DNNs. We explored two kind of deep architectures: a CNN and a ResNet [15], both with and without pretrained weights.

The training loop we used is very similar to the approach described in Section 2.3.2. Training each model on its own, then combining them to be an ensemble.

The architecture of the models was kept the same for all the models in the ensemble, the initialisation of the model's weights was varied as well as the way the training examples were fed to the models. By differing the initialisation of the model's weights and the shuffling of training examples, the models in the ensemble are different enough to obtain a good result [7]. This means bagging is not needed, however, it is implemented, so it can be easily turned on if needed.

During the training, adversarial examples were generated to generalize the models. The loss function we used is the one described in Section 4.2.

We trained 5 models for each ensemble, as suggested by the paper [7], but this could also easily be changed.

For our own convenience, we created a configuration file to easily change the architecture of the models, the way the training examples are fed to the models, some hyperparameters, etc.

4.4.1 Simple CNN

For our custom CNN, we used a architecture which is easily changeable, so we can easily add or remove layers to see the effect on the performance of the model. Generally, the architecture consists of a few "convolutional blocks" followed by a single fully connected layers. A "convolutional block" consists of a convolutional layer, followed by a ReLU activation function and a max pooling layer. The kernel size and padding of the convolutional layers are kept constant at 5 and 2 respectively, as are the kernel size of 2 and stride a stride of 2 for the max pooling layers. The number of channels of the convolutional layers and the number of convolutional blocks can be easily changed in the configuration file.

The first convolutional layer's input channels are fixed at 3 or 4, depending on the input data to include the depth channel or not, while its output channels can be changed. The fully connected layer's output channels are also fixed at 4, two for the predicted x and y coordinates, and two for the predicted variance of those coordinates.

As an example, say we have an architecture with three "convolutional blocks", the first one

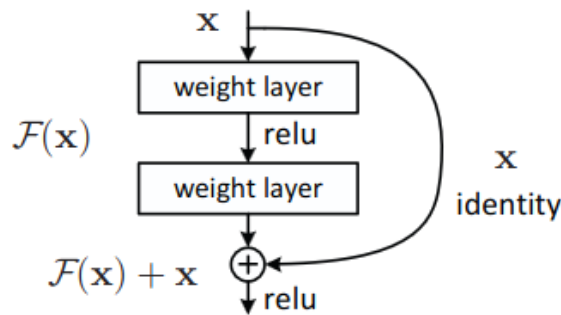


Figure 4.1: A residual block, the main building block of ResNet.

with 8 output channels, the second one with 32, and the third one with 64. After the “convolutional blocks”, the output is flattened and fed to a fully connected layer with 4 output channels.

Comparing architectures We tested a lot of different combinations of architectures and hyperparameters. We compared them using the validation loss on the full ensemble as well as the individual validation losses of the models in the ensemble. Table 4.1 shows the results of some of the architectures we tested.

Index	Architecture	Lr	Br	Epochs	m	Loss
1	[8, 16, 32]	5e-4	1.0	60	5	loss

Table 4.1 Comparison of different architectures for the simple CNN model. “Lr” stands for learning rate, “Br” stands for bagging ratio, “m” stands for the number of models in the ensemble, and “Loss” stands for the ensemble validation loss.

4.4.2 Pretrained ResNet[15]

ResNet [15] is a very popular architecture for image classification tasks, and it has been shown to perform well. The main idea behind ResNet is the use of residual connections (shown in Figure 4.1), which allow the model to learn residual functions instead of direct mappings. This allows the model to learn more complex functions and to avoid the vanishing gradient problem, which is a common problem in deep learning. The architecture of ResNet is based on a series of residual blocks, which consist of two or three convolutional layers with a skip connection between the input and the output of the block. The number of residual blocks and the number of convolutional layers in each block can be varied to create different versions of ResNet, such as ResNet-18, ResNet-34, ResNet-50, etc.

The ResNet architecture we used for our experiments is a ResNet-18, which is quite a small version of ResNet with 8 residual blocks. The architecture, as is, is used for image classification tasks, so we needed to modify it to be used for our regression task. The

main modification we made is to replace the final fully connected layer, which has 1000 output channels for the 1000 classes in ImageNet, with a fully connected layer with 4 output channels.

The PyTorch [13] library provides a convenient implementation of ResNet, along with pre-trained weights. These are weights that have been trained on ImageNet-1K [16], a large dataset of images with 1000 classes. Using pretrained weights can be beneficial for our task, as we don't need to train the full architecture's weights from our own dataset, which would be too small for training such a large architecture. The pretrained weights can be used as a starting point for training our model, and we can fine-tune the model on our own dataset to adapt it to our specific task.

In Table 4.2, we show the results of some of the architectures we tested for the ResNet model, both with and without pretrained weights.

Index	Architecture	Lr	Br	Epochs	m	PW	Loss
1	[8, 16, 32]	0.001	1.0	60	5	Yes	loss

Table 4.2 Comparison of different architectures for the ResNet model. “Lr” stands for learning rate, “Br” stands for bagging ratio, “m” stands for the number of models in the ensemble, “PW” stands for pretrained weights used, and “Loss” stands for the ensemble validation loss.

4.5 MC dropout

MC dropout, as explained in Section 2.3.1, keeps dropout active during inference, creating an ensemble of models. The architecture of the model used for MC dropout is similarly set up as the simple CNN described in Section 4.4.1, also enabling us to easily change the architecture of the model. The main difference is, of course, the addition of dropout layers to the “convolutional blocks”. The dropout rate of each block can easily be changed so we can see the effect of different dropout rates on the performance of the model.

The training loop is a bit different from the one used for deep ensembles, as we only need to train a single model. This model is trained like all the models in a deep ensemble, with the same loss function and the same way of feeding the training examples, adversarial training, etc. There is the difference of having dropout available during training, so not using it would be a missed opportunity to regularize the model, so we keep it active during training as well.

Table 4.3 shows the results of some of the architectures we tested for the MC dropout model.

Index	Architecture	DR	Lr	Br	Epochs	F	Loss
1	[32, 64, 128]	0.1	$1 \cdot 10^{-4}$	1.0	100	100	loss

Table 4.3 Comparison of different architectures for the MC dropout model. “DR” stands for dropout rate at each layer, “Lr” stands for learning rate, “Br” stands for bagging ratio, “F” stands for the number of forward passes, and “Loss” stands for the model’s validation loss.

Chapter 5

Experiments

In this chapter, we will discuss the experiments we conducted to evaluate the performance of our uncertainty-aware object detection model. We will describe the experiments done, present the obtained results, and analyze the performance of the model with a focus on its ability to capture and communicate uncertainty effectively. Finally, we will analyze the results and discuss their implications for the use of uncertainty-aware models in robotic applications.

5.1 Evaluation metrics

To measure how good the model is performing, we need to have a way to see if the model is accurate (the mean prediction is close to the target) and if the uncertainty is communicated well (the uncertainty of the predictions is high when the error is high, and low when the error is low). This is a direct consequence of the chosen loss function, which is supposed to balance these two aspects. There are a few ways we can evaluate the model's performance, each with their own focus and advantages. The datapoints used for evaluation obviously need to be different from the datapoints used for training and validation to avoid data leakage.

5.1.1 Loss Function

The most obvious choice to evaluate a given model, is to use the loss function. This is a great way to have a single number to compare the capability of the model while keeping the balance of the error and uncertainty.

It is the most important metric from a theoretical point of view, as it tells us exactly how well the model is at estimating its own weaknesses and strengths. However, it is not the best metric to use for our use case, as it does not give us the ability to translate the loss into a value which has a clear meaning in our context.

5.1.2 Uncertainty intervals

In the end, the models need to be used as a part of a robotic system without causing any harm to the environment. We thus need a way to evaluate the model's performance in a way that can easily be translated into the real world. Seeing that the model is outputting a mean and a variance, the step to statistical measures of error and uncertainty is quite small. We want to be able to say something about the certainty of the error of the mean prediction. For example, we want to be able to say that we are 95% certain that the error that the mean prediction falls in a certain interval. This means that the model can, with 95% certainty, predict an interval of $[\mu_\theta - \epsilon, \mu_\theta + \epsilon]$ which contains the target value. Note that the model is outputting a mean and a variance, both for the x and y coordinates of the target position, so μ_θ is a 2D vector.

The ϵ , is a product of the σ_θ and the Z-score. The Z-score maps the desired confidence (for example, 95%) level to a standard normal distribution where $\mu = 0$ and $\sigma = 1$. Equation (5.1) shows how to calculate ϵ for a given confidence level α and a variance σ_θ^2 .

$$\epsilon(\alpha, \sigma_\theta^2) = z_\alpha \cdot \sigma_\theta \quad (5.1a)$$

$$\epsilon(\alpha, \sigma_\theta^2) = z_\alpha \cdot \sqrt{\sigma_\theta^2} \quad (5.1b)$$

Note that the variables ϵ , σ_θ , and σ_θ^2 are scalars, this thus holds in one dimension. To go to two dimensions, we need to change one thing, the z_α . In one dimension, the z_α is a single value, but in two dimensions, we need to use a value from the chi-squared distribution with 2 degrees of freedom. Equation (5.2) shows how to calculate ϵ for a given confidence level α and a variance σ_θ^2 .

$$\epsilon(\alpha, \sigma_\theta^2) = \sqrt{\chi_\alpha^2} \cdot \sigma_\theta \quad (5.2a)$$

$$\epsilon(\alpha, \sigma_\theta^2) = \sqrt{\chi_\alpha^2} \cdot \sqrt{\sigma_\theta^2} \quad (5.2b)$$

Let's take a concrete example to make this more clear. The model outputs a mean prediction of $\mu_\theta = [50, 55]mm$ and a variance of $\sigma_\theta^2 = [4, 9]mm^2$. We want to make an interval prediction with a confidence level of 95%. The χ^2 value for 95% confidence level and 2 degrees of freedom is 5.991. Using Equation (5.2), we can calculate ϵ , which comes out to be $\epsilon = [4.90, 7.34]mm$. This then gives us the following interval prediction:

$x \in [50 - 4.90, 50 + 4.90]mm$ and $y \in [55 - 7.34, 55 + 7.34]mm$.

These intervals, give a great way to evaluate the model's performance in a way that can be easily translated into the real world.

5.1.3 Classifying predictions based on error and uncertainty

Using the interval predictions from Section 5.1.2 along with the target value, opens up the possibility to classify the predictions into different categories based on their error and uncertainty. The following situations can occur:

- The model is confident it will succeed in grasping the object, and it does. This is a true positive (TP). This is the ideal situation, as the model is accurate and confident in its predictions.
- The model is not confident it will succeed in grasping the object, but it does. This is a false positive (FP). This is a missed opportunity, as the model is timid
- The model is confident it will succeed in grasping the object, but it does not. This is a false negative (FN). This leads to dangerous situations, as the model is overconfident.
- The model is not confident it will succeed in grasping the object, and it does not. This is a true negative (TN). This is a safe situation, as the model is cautious when it should be.

To actually do this classification, we need to define the *max_error* the model can make while still grabbing the object successfully. This is a value that can be defined based on the specific use case, which in our case is the radiance of the object to be grasped, which is 15mm. This value is then translated, using Equation (5.3), to a *maximum_variance* using the χ^2 value for the desired confidence level and the number of dimensions, which is 95% and 2 respectively in our case.

$$maximum_variance = \frac{max_error^2}{\chi_\alpha^2} \quad (5.3)$$

This gives us all the necessary information to calculate whether a prediction's confidence is not misplaced (Equation (5.4a)), as well as whether the prediction is actually accurate enough (Equation (5.4b)).

$$max(\sigma_\theta^2) \leq maximum_variance \quad (5.4a)$$

$$\sqrt{\sum_{k=1}^d (\mu_k - y_k)^2} \leq max_error \quad (5.4b)$$

If both of these conditions are satisfied, we have a TP. If the first condition is satisfied, but the second condition is not, we have a FP. If the first condition is not satisfied, but the second condition is, we have a FN. If both conditions are not satisfied, we have a TN.

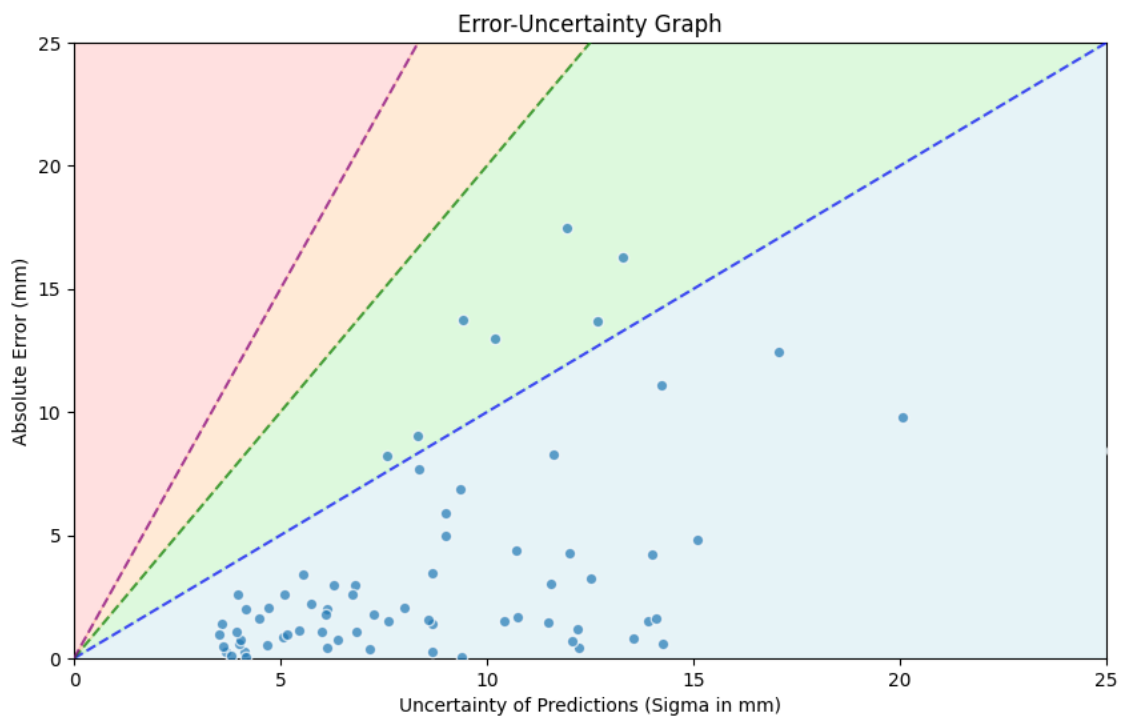


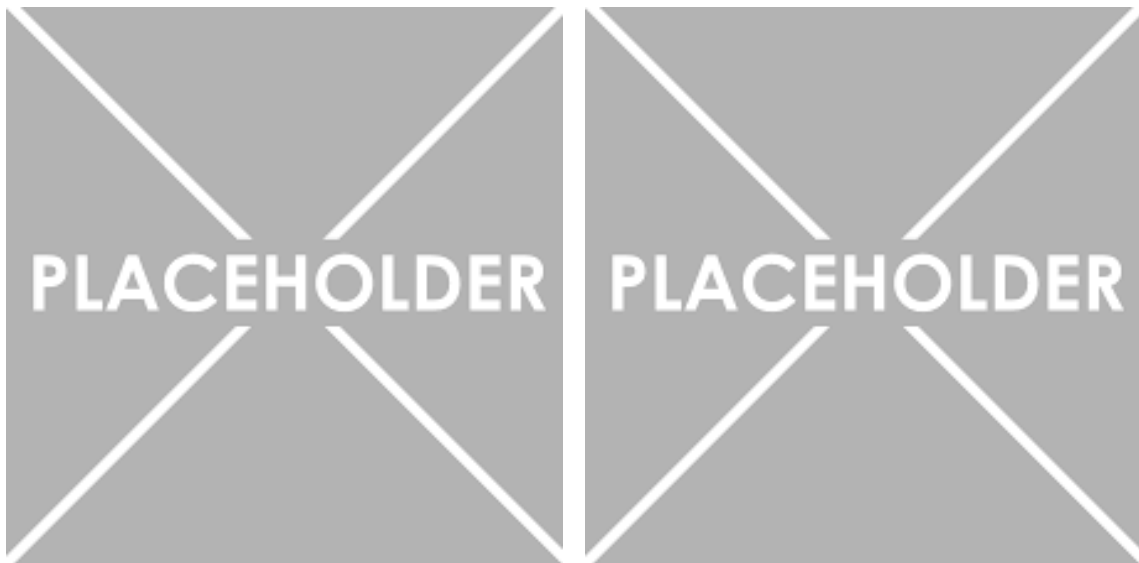
Figure 5.1: An example of an error-uncertainty graph.

5.1.4 Error-uncertainty graph

The statistical scores are well to evaluate and compare single instances given to the models. The loss gives a general idea of how well the model is performing. But both of these metrics fail to give a greater overview of how the model is performing over a greater set of samples. The classification of predictions aids with this, but fails to communicate its hyperparameters and the severity of the errors and uncertainties. A graph, like shown in Figure 5.1, gives a better overview of the models performance over a greater set of samples and the hyperparameters.

The y-axis of the graph shows the uncertainty of a given prediction, this is the $\sqrt{\sigma_{\theta,i}^2}$ of the model's output. The x-axis shows the error made by the model, this is calculated as $y_i - \mu_{\theta,i}$. The graph is divided into 4 zones, each corresponding to a certainty level (α) of the model's predictions:

- Blue zone ($\alpha = 0.68$): It is to be expected that 68% of the predictions fall in this zone, as this is the center of the normal distribution. This is the "Cautious" zone, as the model is underconfident in its predictions.
- Green zone ($\alpha = 0.95$): It is to be expected that 27% of the predictions fall in this zone, as this is the area between the 68% and 95% confidence levels of the normal distribution. This is the "Sweet Spot" zone, as the model is well calibrated in this zone.



(a) Error-uncertainty graph for Deep Ensemble 1.

(b) Error-uncertainty graph for Deep Ensemble 2.

Figure 5.2: Error-uncertainty graphs for the Deep Ensemble models on the test set.

- Orange and red zone ($\lim_{\alpha \rightarrow 1}$): It is to be expected that 5% of the predictions fall in this zone, as this is the area above the 95% confidence level of the normal distribution. This is the "Danger" zone, as the model is blindly confident in its predictions while being way off.

5.2 Experiment 1: Evaluating on a test set

The first test we conducted was to evaluate the model's on a test set. This test set does not differ from the training and validation set in terms of the distribution of the data, but it does differ in terms of the specific samples used. It would be expected that the models perform well on this and perform similarly to how they performed on the validation set.

Model index	Loss	TP	FP	FN	TN
Deep Ensemble ()	loss	TP	FP	FN	TN
ResNet Ensemble ()	loss	TP	FP	FN	TN
MC Dropout ()	loss	TP	FP	FN	TN

Table 5.1 Comparison of the different models on the test set.

5.3 Experiment 2: Evaluating on out-of-distribution data

5.3.1 Multiple objects in the scene

5.3.2 Difficult lighting

5.3.3 Human intervention

Chapter 6

Discussion

6.1

Chapter 7

Conclusion

7.1 Future work

Pixel-coordinates vs real-world coordinates, which more accurately represents the real-world position of the objects? Pixel coordinates as input to the model + post-processing to convert to real-world coordinates. Adaptations to the loss function to see if the too cautious thing can be fixed.

Bibliography

- [1] J. Gawlikowski, C. R. N. Tassi, M. Ali, J. Lee, M. Humt, J. Feng, A. Kruspe, R. Triebel, P. Jung, R. Roscher, M. Shahzad, W. Yang, R. Bamler, and X. X. Zhu, "A survey of uncertainty in deep neural networks", *Artificial Intelligence Review*, vol. 56, no. 1, pp. 1513–1589, Oct. 2023, issn: 1573-7462. doi: 10.1007/s10462-023-10562-9. [Online]. Available: <https://doi.org/10.1007/s10462-023-10562-9>.
- [2] F. K. Gustafsson, M. Danelljan, and T. B. Schon, "Evaluating scalable bayesian deep learning methods for robust computer vision", in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, Jun. 2020.
- [3] O. Ronneberger, P. Fischer, and T. Brox, "U-net: Convolutional networks for biomedical image segmentation", in *Medical Image Computing and Computer-Assisted Intervention—MICCAI 2015: 18th International Conference, Munich, Germany, October 5-9, 2015, Proceedings, Part III 18*, Springer, 2015, pp. 234–241.
- [4] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection", in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 779–788.
- [5] G. Li, L. Yang, C.-G. Lee, X. Wang, and M. Rong, "A bayesian deep learning rul framework integrating epistemic and aleatoric uncertainties", *IEEE Transactions on Industrial Electronics*, vol. 68, no. 9, pp. 8829–8841, 2020.
- [6] M. Verbeke, *Artificial neural networks and deep learning, Part 5: Bayesian learning of neural networks*, Lecture notes, KU Leuven, Course lecture, 2025.
- [7] B. Lakshminarayanan, A. Pritzel, and C. Blundell, "Simple and scalable predictive uncertainty estimation using deep ensembles", in *Advances in Neural Information Processing Systems*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds., vol. 30, Curran Associates, Inc., 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/file/9ef2ed4b7fd2c810847ffa5fa85bce38-Paper.pdf.
- [8] Y. Gal and Z. Ghahramani, "Dropout as a bayesian approximation: Representing model uncertainty in deep learning", in *Proceedings of The 33rd International Conference on Machine Learning*, M. F. Balcan and K. Q. Weinberger, Eds., ser. Proceedings of Machine Learning Research, vol. 48, New York, New York, USA: PMLR, 2016, pp. 1050–1059. [Online]. Available: <https://proceedings.mlr.press/v48/gal16.html>.
- [9] Eran, S. Dorodnicov, N. Azkiel, remibettan, aangerma, S. Orsten, O. Meir, ev-mp, Noha Yassin, D. Diakopoulos, Z. Shahaf, A. Avraham, M. Katz, I. Carpis, noacoochen, Tamir, MMirbach, S. Khshiboun, doronhi, AnnaRomanov, gwen2018, J. Radford, Noy-Zini, alowenst01, A. Bernstein, D. Tong, B. Fulkerson, T. Shao, gilpazintel, and shiritbrook, *Realsenseai/librealsense*, <https://github.com/realsenseai/librealsense>, Mar. 2026. [Online]. Available: <https://github.com/realsenseai/librealsense>.

- [10] ABB Robotics, *Product specification: Irb 1200 (on omnicore)*, Document ID: 3HAC081417-001, Revision: L, ABB Robotics Sweden AB, S-721 71 VÄSTERÅS, Sweden, Nov. 2025. [Online]. Available: <https://new.abb.com/products/robotics>.
- [11] M. De Ryck, *Robot_demonstrator: Software for the robot demonstrator at ku leuven bruges*, https://github.com/MatthiasDR96/robot_demonstrator, ABB IRB1200 robot control with RealSense D415 and SICK Microscan 3, Dec. 2024.
- [12] G. Bradski, *The OpenCV Library*, 2000.
- [13] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "Pytorch: An imperative style, high-performance deep learning library", in *Advances in Neural Information Processing Systems 32*, Curran Associates, Inc., 2019, pp. 8024–8035. [Online]. Available: <http://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.pdf>.
- [14] S. O. Contributors, *How to detect lines in OpenCV?*, <https://stackoverflow.com/questions/45322630/how-to-detect-lines-in-opencv>, Online; accessed 15-April-2026, 2017.
- [15] K. He, X. Zhang, S. Ren, and S. Jian, "Deep residual learning for image recognition", in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Jun. 2016, pp. 770–778.
- [16] O. Russakovsky et al., "ImageNet Large Scale Visual Recognition Challenge", *IJCV*, vol. 115, no. 3, pp. 211–252, 2015.

Chapter 8

GenAI code of conduct for students

Generative AI (GenAI) assistance tools can be used to generate text, image, code, video, music, or combinations of these. It includes typical tools like (but this list is not limited to): ChatGPT, Google Gemini, MS Copilot, Midjourney, Claude.ai, Perplexity.ai, Dall-E, etc.

Student Information

Student name: Thijs Alens

This form is related to my Master thesis.

Title Master thesis: Modeling object localization uncertainty for robot grasping Pro-
moter: _____

Daily supervisor: _____

Use of GenAI Assistance

Please indicate with "X" (possibly multiple times) in which way you were using GenAI:

☐ I did not use any GenAI assistance tool.

☐ I did use GenAI assistance. In this case, specify which ones (e.g., ChatGPT, ...):

GenAI Assistance Used As/For

☐ MS Copilot

- ☐ Language assistance
- ☐ Search engine
- ☐ Literature search
- ☐ Short-form input assistance
- ☐ Generating programming code
- ☐ Generating new research ideas
- ☐ Generating blocks of text
- ☐ Other (specify): _____

Code of Conduct for Different Uses

Language Assistant for Reviewing or Improving Texts

This use is similar to using spelling and grammar check tools, and you do not have to refer to the use of GenAI in the text. Be careful:

- Using GenAI tools on texts you did not write yourself to cover up plagiarism (by paraphrasing original texts) is not allowed.

Search Engine for Information or Existing Research

This use is similar to a Google search or checking Wikipedia. If you then write your own text based on this information, you do not have to refer to the use of GenAI in the text. Be careful:

- Be aware that the output of the GenAI tool cannot be guaranteed as a 100% reliable source of information.
- The output may not be entirely correct and is limited due to the databases it uses. Knowledge evolves and may change over time, and it may be that the database of the GenAI tool is not up to date.

Literature Search

This use is comparable to a Google Scholar search. Be careful:

- Be aware that the output is restricted to the database it is built on. After this initial search, look for scientific sources and conduct your own analysis.
- GenAI tools (like ChatGPT) may output no or wrong references. As a student, you are responsible for further checking and verifying the accuracy of references.

Short-form Input Assistance

This use is similar to Google Docs powered by generative language models.

Generating Programming Code

The use of GenAI for coding should be explicitly allowed by the teacher. If used for coding, correctly mention the use of GenAI assistance and cite it.

Generating New Research Ideas

Further verify whether the idea is novel or not. It is likely that it is related to existing work, which should be referenced.

Generating Blocks of Text

Inserting blocks of text without quotes and a reference to GenAI assistance in your report or thesis is not allowed. Be careful:

- When you literally copy elements from a conversation with a GenAI tool, quote them between quotation marks and refer to them according to the specified reference style.
- Describe the use of the GenAI tool (tool name, version, date, etc.) in the method section and optionally add the full conversation as an attachment.

Other

Contact the professor of the course or the promoter of the thesis. Inform also the program director. Motivate how you comply with Article 84 of the exam regulations. Explain the use and added value of ChatGPT or another AI tool.

Further Important Guidelines and Remarks

- GenAI assistance cannot be used related to data or subjects under Non-Disclosure Agreement.
- GenAI assistance cannot be used related to sensitive or personal data due to privacy issues.
- Take a scientific and critical attitude when interacting with GenAI assistance and interpreting its output.
- As a student, you are responsible for complying with Article 84 of the exam regulations: your report or thesis should reflect your own knowledge, understanding, and skills.

Exam Regulations Article 84

“Every conduct individual students display with which they (partially) inhibit or attempt to inhibit a correct judgement of their own knowledge, understanding and/or skills or those of other students, is considered an irregularity which may result in a suitable penalty. A special type of irregularity is plagiarism, i.e., copying the work (ideas, texts, structures, designs, images, plans, codes, . . .) of others or prior personal work in an exact or slightly modified way without adequately acknowledging the sources.”

Additional Reading

More information about being transparent on the use of GenAI assistance and about citing and referencing GenAI can be found on the student website.

A Few Final Words

If you are uncertain whether or not you should declare your use of AI tools, discuss the matter with your teacher or promoter. It is safer to declare AI use when it is not needed than to withhold that declaration when it is required.

Finally, remember that advanced AI tools are new and that they can do things they could not do until recently. It is important to follow up on the most recent developments in AI technologies and communicate openly with your teachers, assistants, supervisors, and peers.